Assignment 04

Question 01

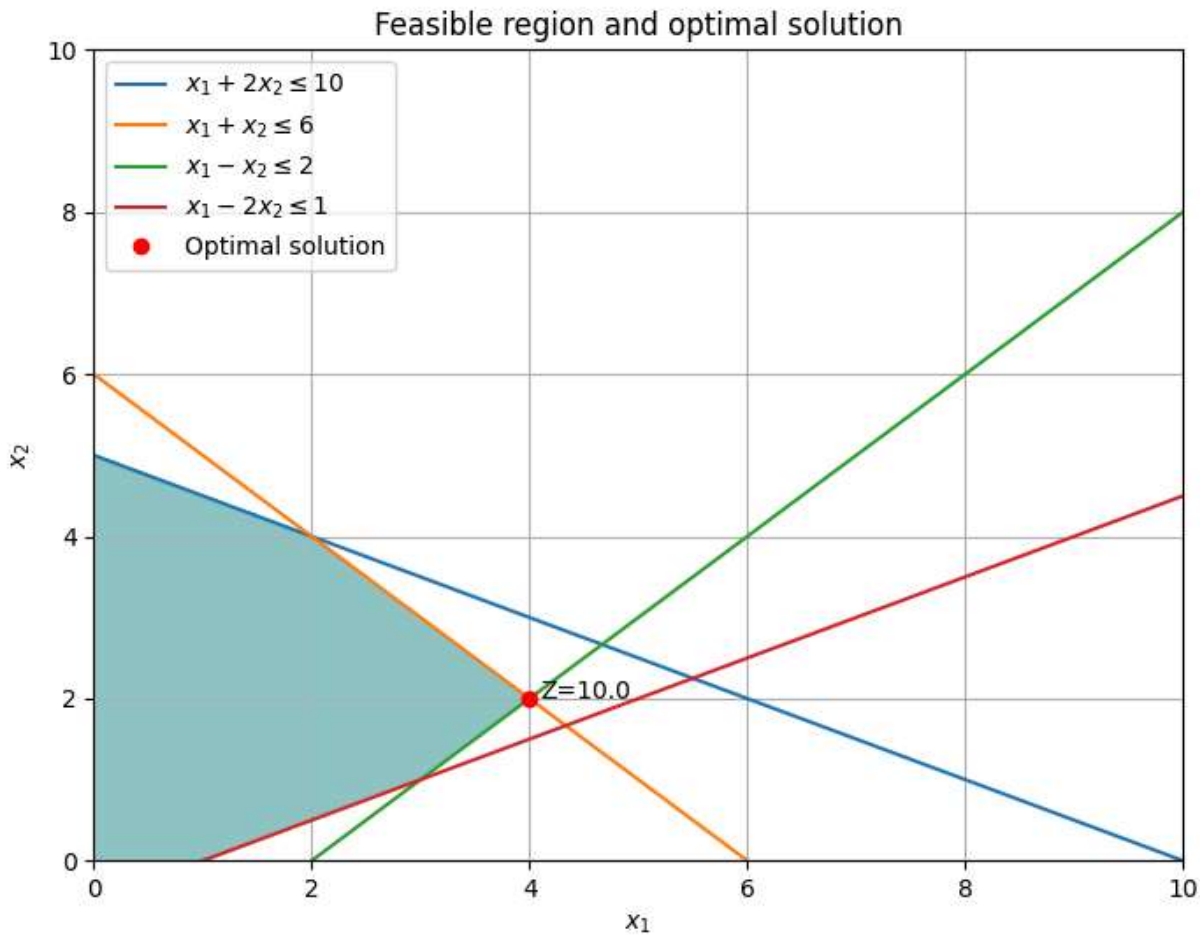
In [21]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import linprog
#Define the objective function
c=[-2,-1] #Negated for maximization
#Constraints
A=[[1,2],[1,1],[1,-1],[1,-2]]
b=[10,6,2,1]
#Bounds
x0_bounds=(0,None)
x1_bounds=(0,None)
#Solve using linprog
res=linprog(c,A_ub=A,b_ub=b,bounds=[x0_bounds,x1_bounds],method='highs')

print('Optimal Solution:')
print(f'x1={res.x[0]}')
print(f'x2={res.x[1]}')
print(f'Maximum Z={-res.fun}')
#plotting
x=np.linspace(0,10,400)
plt.figure(figsize=(8,6))
#Plot each constraint line
plt.plot(x,(10-x)/2,label=r'$x_1+2x_2\leq 10$')
plt.plot(x,6-x,label=r'$x_1+x_2\leq 6$')
plt.plot(x,x-2,label=r'$x_1-x_2\leq 2$')
plt.plot(x,(x-1)/2,label=r'$x_1-2x_2\leq 1$')
#Fill feasible region
x1=np.linspace(0,10,400)
x2=np.linspace(0,10,400)
X1,X2=np.meshgrid(x1,x2)

ineq1=(X1+2*X2<=10)
ineq2=(X1+X2<=6)
ineq3=(X1-X2<=2)
ineq4=(X1-2*X2<=1)
ineq5=(X1>=0)&(X2>=0)
feasible=ineq1&ineq2&ineq3&ineq4&ineq5
plt.contourf(X1,X2,feasible,levels=[0.5,1],alpha=0.5)
#Plot optimal point
if res.success:
    plt.plot(res.x[0],res.x[1],'ro',label='Optimal solution')
    plt.text(res.x[0]+0.1,res.x[1],f'Z={round(-res.fun,2)}')
#Labeling
plt.xlim(0,10)
plt.ylim(0,10)
plt.xlabel(r'$x_1$')
plt.ylabel(r'$x_2$')
plt.title('Feasible region and optimal solution')
```

```
plt.legend()
plt.grid()
plt.show()
```

```
Optimal Solution:
x1=4.0
x2=2.0
Maximum Z=10.0
```



Question 02

In [22]:
```
from scipy.optimize import linprog
#Define the objective function
c=[45,40,85,65]
#Constraints
A=[[-3,-4,-8,-6], #Proteins
   [-2,-2,-7,-5], #Fats
   [-6,-4,-7,-4]  #Carbohydrates
   ]
b=[-800,-200,-700]
#Bounds
x_bounds=[(0,None)]*4
#Solve using linprog
res=linprog(c,A_ub=A,b_ub=b,bounds=x_bounds,method='highs')

#Output result
if res.success:
```

```
    print('Food units to purchase',res.x)
    print(f'Minimum cost: {res.fun} BDT')
```

```
Food units to purchase [  0. 200.   0.   0.]
Minimum cost: 8000.0 BDT
```

Question 03

In [23]:

```
import numpy as np
def simplex_solve():
    tableau=np.array([
        [  1.0,   1.0,   1.0,  1.0,  0.0,  0.0, 100.0],  # s1 constraint
        [  0.02,  0.04,  0.03, 0.0,  1.0,  0.0,   3.0],  # s2 constraint
        [  3.0,   2.0,   5.0,  0.0,  0.0,  1.0, 300.0],  # s3 constraint
        [-12.0, -15.0, -14.0,  0.0,  0.0,  0.0,   0.0]   # objective row
    ])
    variables=['s1','s2','s3']  # Basic variables
    iteration=0
    while True:
        iteration+=1
        print(f"ITERATION {iteration}")
        print(f"{'-'*50}")
        # Print current tableau
        print("\nCurrent Tableau:")
        headers=['x1','x2','x3','s1','s2','s3','RHS']
        print(f"{'Var':<4}", end="")
        for h in headers: print(f"{h:>8}", end="")
        print()

        for i in range(3):
            print(f"{variables[i]:<4}", end="")
            for j in range(7): print(f"{tableau[i,j]:>8.2f}", end="")
            print()

        print(f"{'Z':<4}", end="")
        for j in range(7): print(f"{tableau[3,j]:>8.2f}", end="")
        print()

        # Check optimality
        if np.all(tableau[3,:-1]>=0):
            print(f"\nOptimal solution reached!")
            break

        # Find entering variable (most negative in objective row)
        entering_col=np.argmin(tableau[3,:-1])
        entering_var=headers[entering_col]
        print(f"\nEntering variable: {entering_var}")

        # Find leaving variable (minimum ratio test)
        ratios=[]
        for i in range(3):
            if tableau[i,entering_col]>0:
                ratios.append(tableau[i,-1]/tableau[i,entering_col])
            else:
                ratios.append(float('inf'))
        leaving_row=np.argmin(ratios)
```

```
        leaving_var=variables[leaving_row]
        print(f"Outgoing variable: {leaving_var}")

        # Pivot operation
        pivot=tableau[leaving_row,entering_col]
        tableau[leaving_row] /=pivot  # Normalize pivot row

        # Eliminate other entries in pivot column
        for i in range(4):
            if i != leaving_row:
                tableau[i] -= tableau[i, entering_col] * tableau[leaving_row]

        # Update basic variables
        variables[leaving_row] = entering_var

    # Extract solution
    solution = np.zeros(3)
    for i, var in enumerate(variables):
        if var in ['x1', 'x2', 'x3']:
            var_index = ['x1', 'x2', 'x3'].index(var)
            solution[var_index] = tableau[i, -1]

    profit = -tableau[3, -1]

    print(f"\n{'='*50}")
    print("FINAL SOLUTION")
    print(f"{'='*50}")
    print(f"Coal A (x1): {solution[0]:.0f} tons")
    print(f"Coal B (x2): {solution[1]:.0f} tons")
    print(f"Coal C (x3): {solution[2]:.0f} tons")
    print(f"Maximum Profit: {-profit:.0f} BDT")

    return solution, profit

# Run the solver
solution, profit = simplex_solve()
```

```
ITERATION 1
--------------------------------------------------

Current Tableau:
Var        x1      x2      x3      s1      s2      s3     RHS
s1       1.00    1.00    1.00    1.00    0.00    0.00  100.00
s2       0.02    0.04    0.03    0.00    1.00    0.00    3.00
s3       3.00    2.00    5.00    0.00    0.00    1.00  300.00
Z      -12.00  -15.00  -14.00    0.00    0.00    0.00    0.00

Entering variable: x2
Outgoing variable: s2
ITERATION 2
--------------------------------------------------

Current Tableau:
Var        x1      x2      x3      s1      s2      s3     RHS
s1       0.50    0.00    0.25    1.00  -25.00    0.00   25.00
x2       0.50    1.00    0.75    0.00   25.00    0.00   75.00
s3       2.00    0.00    3.50    0.00  -50.00    1.00  150.00
Z       -4.50    0.00   -2.75    0.00  375.00    0.00 1125.00

Entering variable: x1
Outgoing variable: s1
ITERATION 3
--------------------------------------------------

Current Tableau:
Var        x1      x2      x3      s1      s2      s3     RHS
x1       1.00    0.00    0.50    2.00  -50.00    0.00   50.00
x2       0.00    1.00    0.50   -1.00   50.00    0.00   50.00
s3       0.00    0.00    2.50   -4.00   50.00    1.00   50.00
Z        0.00    0.00   -0.50    9.00  150.00    0.00 1350.00

Entering variable: x3
Outgoing variable: s3
ITERATION 4
--------------------------------------------------

Current Tableau:
Var        x1      x2      x3      s1      s2      s3     RHS
x1       1.00    0.00    0.00    2.80  -60.00   -0.20   40.00
x2       0.00    1.00    0.00   -0.20   40.00   -0.20   40.00
x3       0.00    0.00    1.00   -1.60   20.00    0.40   20.00
Z        0.00    0.00    0.00    8.20  160.00    0.20 1360.00

Optimal solution reached!

==================================================
FINAL SOLUTION
==================================================
Coal A (x1): 40 tons
Coal B (x2): 40 tons
Coal C (x3): 20 tons
Maximum Profit: 1360 BDT
```

Question 04

In [24]:
```
import numpy as np
def big_m_method():
    # Columns: x, y, s1, s2, s3, a1, a2, a3
    A=np.array([
        [200, 100, -1, 0, 0, 1, 0, 0],      # Vitamins
        [  1,   2,  0,-1, 0, 0, 1, 0],      # Minerals
        [ 40,  40,  0, 0,-1, 0, 0, 1]       # Calories
    ])

    b=np.array([4000, 50, 1400])
    M=1e4  # Large positive number
    c=np.array([4, 3, 0, 0, 0, M, M, M])
    basic_vars=[5, 6, 7]  # Initial basic variables: a1, a2, a3 (indices 5,6,7)

    # Initial tableau: [A | b]
    tableau=np.hstack((A, b.reshape(-1, 1)))

    artificial_rows = A[[0,1,2], :]  # Rows corresponding to a1, a2, a3
    obj_row=c-M*np.sum(artificial_rows, axis=0)  # Z=c-M*sum of artificial rows
    tableau=np.vstack((tableau, np.append(obj_row, 0)))

    var_names=['x','y','s1','s2','s3','a1','a2','a3']  # Variable names for printing

    iteration=0
    print("=" * 100)
    print("Big-M method simplex iterations")
    print("=" * 100)

    # Simplex iterations
    while True:
        iteration += 1

        # Print current basic variables
        basic_vars_names=[var_names[i] for i in basic_vars]
        print(f"Iteration -->{iteration:4} | Basic Variables -->{str(basic_vars_names

        # Print tableau header
        print(f"\nTableau {iteration}:")
        print(f"{'':>12} " + " ".join(f"{name:>10}" for name in var_names) + f"{'RHS'
        print("-" * 113)

        # Print constraint rows
        constraint_names=['Vitamins', 'Minerals', 'Calories', 'Objective']
        for i, row in enumerate(tableau):
            row_name=constraint_names[i] if i < len(constraint_names) else f"Row {i}'
            print(f"{row_name:>12} " + " ".join(f"{v:>10.1f}" for v in row))
        print()

        # Find the entering variable (most negative in obj row)
        obj_row=tableau[-1, :-1]
        if np.all(obj_row >= 0):
            print("OPTIMAL SOLUTION REACHED - All coefficients in objective row are r
            break  # Optimal solution reached
```

```
        entering=np.argmin(obj_row)

        # Find the leaving variable (min ratio test)
        ratios=tableau[:-1, -1] / tableau[:-1, entering]
        ratios[ratios <= 0]=np.inf
        leaving=np.argmin(ratios)

        # Print entering and leaving variables
        print(f"→ Entering variable: {var_names[entering]} (most negative coefficient
        print(f"→ Leaving variable: {var_names[basic_vars[leaving]]} (minimum ratio t
        print(f"→ Pivot element: {tableau[leaving, entering]:.3f}")
        print()

        pivot_element=tableau[leaving, entering]  # Pivot
        tableau[leaving, :] /= pivot_element
        for i in range(tableau.shape[0]):
            if i != leaving:
                factor=tableau[i, entering]
                tableau[i, :] -=factor*tableau[leaving, :]

        # Update basic variables
        basic_vars[leaving]=entering

        print("After pivoting:")
        print("-" * 60)

    # Extract solution
    solution=np.zeros(len(c))
    for i, var in enumerate(basic_vars):
        if var < len(c):
            solution[var]=tableau[i, -1]

    x,y=solution[0],solution[1]
    total_cost=4*x+3*y
    return x, y, total_cost

# Run the function
x,y,total_cost=big_m_method()

print("\n" + "=" * 50)
print("Final Solution")
print("=" * 50)
print(f"Optimal number of units of Food A: {x}")
print(f"Optimal number of units of Food B: {y}")
print(f"Total Cost (BDT): {total_cost}")
```

```
==========================================================================================
==============
Big-M method simplex iterations
==========================================================================================
==============
Iteration -->   1 | Basic Variables -->['a1', 'a2', 'a3']

Tableau 1:
                      x          y         s1         s2         s3         a1
a2         a3         RHS
------------------------------------------------------------------------------------------
---------------------------
    Vitamins      200.0      100.0       -1.0        0.0        0.0        1.0
0.0        0.0     4000.0
    Minerals        1.0        2.0        0.0       -1.0        0.0        0.0
1.0        0.0       50.0
    Calories       40.0       40.0        0.0        0.0       -1.0        0.0
0.0        1.0     1400.0
   Objective -2409996.0 -1419997.0    10000.0    10000.0    10000.0        0.0
0.0        0.0        0.0

→ Entering variable: x (most negative coefficient)
→ Leaving variable: a1 (minimum ratio test)
→ Pivot element: 200.000

After pivoting:
------------------------------------------------------------
Iteration -->   2 | Basic Variables --> ['x', 'a2', 'a3']

Tableau 2:
                      x          y         s1         s2         s3         a1
a2         a3         RHS
------------------------------------------------------------------------------------------
---------------------------
    Vitamins        1.0        0.5       -0.0        0.0        0.0        0.0
0.0        0.0       20.0
    Minerals        0.0        1.5        0.0       -1.0        0.0       -0.0
1.0        0.0       30.0
    Calories        0.0       20.0        0.2        0.0       -1.0       -0.2
0.0        1.0      600.0
   Objective        0.0  -214999.0    -2050.0    10000.0    10000.0    12050.0
0.0        0.0 48199920.0

→ Entering variable: y (most negative coefficient)
→ Leaving variable: a2 (minimum ratio test)
→ Pivot element: 1.500

After pivoting:
------------------------------------------------------------
Iteration -->   3 | Basic Variables -->  ['x', 'y', 'a3']

Tableau 3:
                      x          y         s1         s2         s3         a1
a2         a3         RHS
------------------------------------------------------------------------------------------
---------------------------
```

```
    Vitamins        1.0        0.0       -0.0        0.3        0.0        0.0        -
0.3        0.0       10.0
    Minerals        0.0        1.0        0.0       -0.7        0.0       -0.0
0.7        0.0       20.0
    Calories        0.0        0.0        0.1       13.3       -1.0       -0.1       -1
3.3        1.0      200.0
   Objective        0.0        0.0    -1333.3  -133332.7    10000.0    11333.3    14333
2.7        0.0 52499900.0

→ Entering variable: s2 (most negative coefficient)
→ Leaving variable: a3 (minimum ratio test)
→ Pivot element: 13.333

After pivoting:
------------------------------------------------------------
Iteration -->   4 | Basic Variables -->  ['x', 'y', 's2']

Tableau 4:
                     x          y         s1         s2         s3         a1
a2         a3         RHS
------------------------------------------------------------------------------------
---------------------------
    Vitamins        1.0        0.0       -0.0        0.0        0.0        0.0
0.0       -0.0        5.0
    Minerals        0.0        1.0        0.0        0.0       -0.1       -0.0
0.0        0.1       30.0
    Calories        0.0        0.0        0.0        1.0       -0.1       -0.0       -
1.0        0.1       15.0
   Objective        0.0        0.0        0.0        0.0        0.0    10000.0    1000
0.0    10000.0 54499890.0

OPTIMAL SOLUTION REACHED - All coefficients in objective row are non-negative

==================================================
Final Solution
==================================================
Optimal number of units of Food A: 5.0
Optimal number of units of Food B: 30.0
Total Cost (BDT): 110.0
```

Question 05

In [1]:

```python
import numpy as np
#Supply & Demand
supply=[80,60,40,20]
demand=[60,60,30,40,10]
#Cost matrix
cost=np.array([
   [4,3,1,2,6],
   [5,2,3,4,5],
   [3,5,6,3,2],
   [2,4,4,5,3]
],dtype=float)

def northwest_corner(s,d):
```

```
    s,d=s[:],d[:]
    alloc=np.zeros((len(s),len(d)),dtype=int)
    i=j=0
    while i<len(s) and j<len(d):
        m=min(s[i],d[j])
        alloc[i][j]=m
        s[i]-=m
        d[j]-=m
        if s[i]==0: i+=1
        elif d[j]==0: j+=1
    return alloc

def least_cost(s,d,c):
    s,d=s[:],d[:]
    c=c.copy()
    alloc=np.zeros_like(c)
    while np.any(s) and np.any(d):
        i,j=divmod(np.argmin(c),c.shape[1])
        m=min(s[i],d[j])
        alloc[i][j]=m
        s[i]-=m
        d[j]-=m
        if s[i]==0: c[i,:]=np.inf
        elif d[j]==0: c[:,j]=np.inf
    return alloc

def total_cost(a,c):
    return np.sum(a*c)

nw=northwest_corner(supply,demand)
lc=least_cost(supply,demand,cost)

print('North-West Corner:\n',nw)
print('Cost:',total_cost(nw,cost))
print('\nLeast Cost Method:\n',lc)
print('Cost:',total_cost(lc,cost))
```

```
North-West Corner:
 [[60 20  0  0  0]
 [ 0 40 20  0  0]
 [ 0  0 10 30  0]
 [ 0  0  0 10 10]]
Cost: 670.0

Least Cost Method:
 [[10.  0. 30. 40.  0.]
 [ 0. 60.  0.  0.  0.]
 [30.  0.  0.  0. 10.]
 [20.  0.  0.  0.  0.]]
Cost: 420.0
```

Question 06

In [26]:
```
from pulp import *

# Define supply, demand, and costs
```

```
supply = {'F1': 200, 'F2': 160, 'F3': 90}
demand = {'W1': 180, 'W2': 120, 'W3': 150}
costs = {
    'F1': {'W1': 16, 'W2': 20, 'W3': 12},
    'F2': {'W1': 14, 'W2': 8, 'W3': 18},
    'F3': {'W1': 26, 'W2': 24, 'W3': 16}
}

# Create the linear programming problem
prob = LpProblem("Transportation_Problem", LpMinimize)

# Define decision variables
vars = LpVariable.dicts("Route", (supply.keys(), demand.keys()), 0, None, LpContinuou

# Set the objective function (minimize total cost)
prob += lpSum([vars[f][w] * costs[f][w] for f in supply for w in demand]), "Total Cos

# Add supply constraints
for f in supply:
    prob += lpSum([vars[f][w] for w in demand]) == supply[f], f"Supply_{f}"

# Add demand constraints
for w in demand:
    prob += lpSum([vars[f][w] for f in supply]) == demand[w], f"Demand_{w}"

# Solve the problem
prob.solve()

# Print the results
print("Status:", LpStatus[prob.status])
print("\nOptimal Shipping Plan:")
for v in prob.variables():
    if v.varValue > 0:
        print(f"{v.name.replace('_', ' to ')}: {v.varValue} units")
print(f"\nTotal Shipping Cost: {value(prob.objective)} BDT")
```

```
Status: Optimal

Optimal Shipping Plan:
Route to F1 to W1: 140.0 units
Route to F1 to W3: 60.0 units
Route to F2 to W1: 40.0 units
Route to F2 to W2: 120.0 units
Route to F3 to W3: 90.0 units

Total Shipping Cost: 5920.0 BDT
```

Question 07

In [27]:

```
import numpy as np
from scipy.optimize import linear_sum_assignment
#Profit matrix
profit=np.array([
    [16,10,14,11],
    [14,11,15,15],
    [15,15,13,12],
```

```
    [13,12,14,15]
])
#For maximization
cost=profit.max()-profit
#Solve using Hungarian algorithm
row_ind,col_ind=linear_sum_assignment(cost)
#Output result
total_profit=profit[row_ind,col_ind].sum()
print('Optimal Assignment:')
for i in range(len(row_ind)):
    print(f'Salesman {chr(65+row_ind[i])} \u2192 City {col_ind[i]+1} (profit:{profit[

print('Total maximum profit',total_profit)
```

```
Optimal Assignment:
Salesman A → City 1 (profit:16)
Salesman B → City 3 (profit:15)
Salesman C → City 2 (profit:15)
Salesman D → City 4 (profit:15)
Total maximum profit 61
```

Question 08

In [28]:
```
import numpy as np
from scipy.optimize import linprog
# Payoff matrix for Player A (column player)
A=np.array([
    [-1,-2, 8],
    [ 7, 5,-1],
    [ 6, 0,12]
])
# Since linprog solves minimization problems,
# we will convert the game into a minimization problem for the column player.
# Step 1: Make the payoff matrix positive by adding a constant (if needed)
min_value=np.min(A)
if min_value<0:
    A=A-min_value  # shift matrix to make all elements non-negative
else:
    min_value=0  # no shift needed
# Step 2: Set up the LP
c=[1,1,1]   # Objective: minimize sum of probabilities (due to duality)
A_ub=-A.T     # Constraints: -A^T x <= -1 (because we maximize in original)
b_ub=-np.ones(3)
# Step 3: Solve
res=linprog(c, A_ub=A_ub, b_ub=b_ub, method='highs')
# Step 4: Get the probabilities
probabilities=res.x /np.sum(res.x)
# Step 5: Calculate the value of the game
value=1 /np.sum(res.x)
value=value+min_value  # Adjust back if shifted earlier
print("Optimal mixed strategy for Player A:")
print(f"Play Row 1 with probability {probabilities[0]:.4f}")
print(f"Play Row 2 with probability {probabilities[1]:.4f}")
print(f"Play Row 3 with probability {probabilities[2]:.4f}")

print(f"\nValue of the game: {value:.4f}")
```

```
Optimal mixed strategy for Player A:
Play Row 1 with probability 0.0000
Play Row 2 with probability 0.6667
Play Row 3 with probability 0.3333

Value of the game: 3.3333
```